



Chapter 1

Software Architecture and design

Getting started

Instructor: Abed Safadi, Ph.D.



Plan

- Introduction
- Software Architecture
- Layered Architecture
- Design patterns

References

- Richard N. Taylor, Nenad Medvidovic, Eric M. Dashofy: Software Architecture Foundations, Theory, and Practice.
- Marck Richards: Software Architecture Patterns.
- Erich Gamma, Richard Helm, Ralph Johnson & John Vlissides : Design Patterns : Elements of Reusable Object-Oriented Software.
- Robert C. Martin : Clean Architecture : A Craftsman's Guide to Software Structure and Design
- Websites notes

Introduction

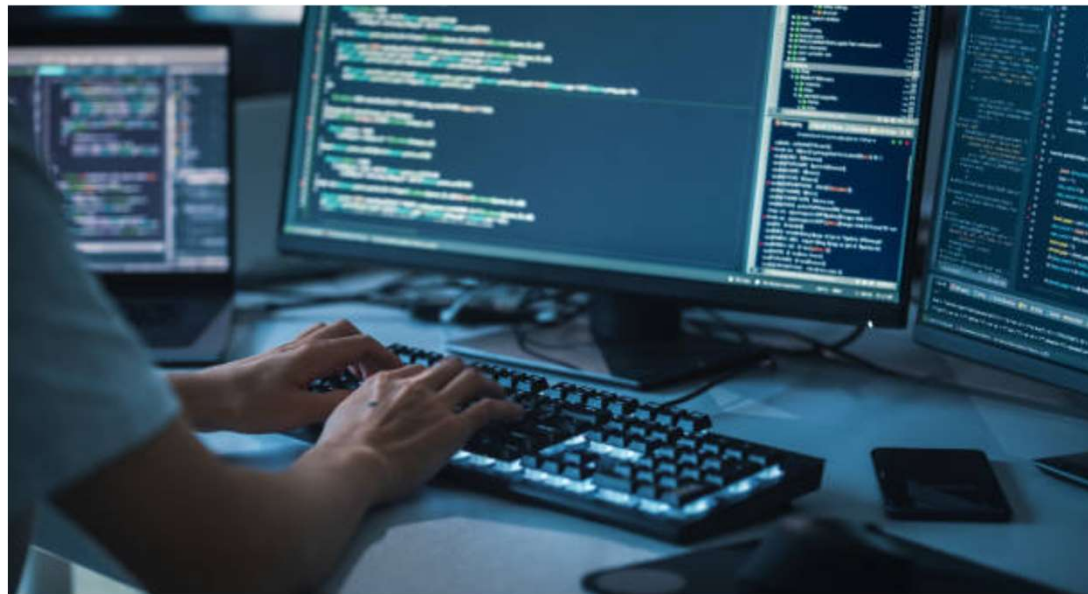


Introduction



Introduction: Architect Job

It's the job of Architect to join both art and science to make sure all the pieces of a building come together in a good solution . Similar to architects as a software engineer you will also need to mix art and science to deliver solutions in a satisfactory way but instead of bricks you will solve them with code.



Introduction

- In the software architecture we focus more on the structure more than implementation details.
- Software architecture refers to the high-level structure and organization of a software system

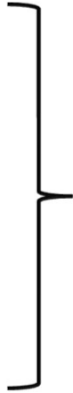
Building an eCommerce Site

What should the system do?

Building an eCommerce Site

What should the system do?

- Search Inventory
- Buy products
- check reviews
- review past orders
-



Functional requirements

Building an eCommerce Site

How should the system behave (ilities)?

Building an eCommerce Site

How should the system behave ?

- Reliability
- Usability
- Efficiency (Short response)
- Maintainability
- Scalability (Able to serve millions of users)
- Availability (24/7)
-



Non-Functional requirements

Finally, besides functional and non-functional requirements you may also have additional **restrictions** that will limit the options that you will have for your architecture (legal compliance, cost, time to market, standards,...)

Non-Functional requirements

How do non-functional requirements (e.g., performance, security, reliability) influence software architecture decisions?

Non-Functional requirements

How do non-functional requirements (e.g., performance, security, reliability) influence software architecture decisions?

- **Performance:** Performance requirements determine how fast the system must respond to user requests and process data. To meet performance requirements, architects may need to make decisions about the **choice of hardware, database design, caching strategies**, and load balancing techniques. They may also consider parallelism and distributed computing to improve performance.
- **Security:** Architects must consider security at every level of the architecture. This **includes authentication mechanisms, access control, encryption**, and protection against common security threats.
- **Reliability:** Reliability requirements dictate how dependable the system must be. This involves minimizing downtime, ensuring fault tolerance,... Architects may design **redundant components, implement backup and recovery mechanisms**, and use clustering to meet reliability goals.
- **Availability:** Availability requirements define how often the system should be operational. Architects need to consider factors like system redundancy, **and disaster recovery planning to ensure high availability**.
- **Cost:** The cost of implementing and maintaining the architecture is another non-functional factor. Architects may need to make decisions that **optimize costs while meeting other NFRs**. For example, cloud-based solutions might be chosen to reduce infrastructure costs.

What is the software architecture?

- A **software architecture** is a description of the **subsystems** and components of a **software system** and the relationships between them.
- **Subsystems** and components are typically specified in different views to show the relevant functional and non-functional properties of a software system.
- The **software system** is the result of the software design activity.

What is the software architecture?

- A **set of principal design decisions** of a software system
- It governs
 - Structure
 - Behavior
 - Interaction
 - Non-functional properties

Component

- A component is an encapsulated part of a software system. A component has an interface.
- Components serve as the building blocks for the structure of a system.
- At a programming-language level, components may be represented as modules, classes, objects or as a set of related functions

Subsystems

- A subsystem is a set of collaborating components performing a given task.
- A subsystem is considered a separate entity within a software architecture.
- It performs its designated task by interacting with other subsystems and components...

Architectural Style

Architectural style in software development refers to a recurring solution or design approach used to solve common problems when designing and organizing the overall structure of a software system

It provides a set of predefined subsystems, their responsibilities, and includes rules and guidelines for organizing the relationships between them.

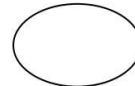
Example

Applying Layer Architecture

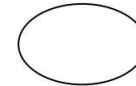
UI



Core

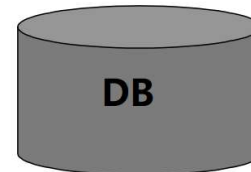


Play



View High Score

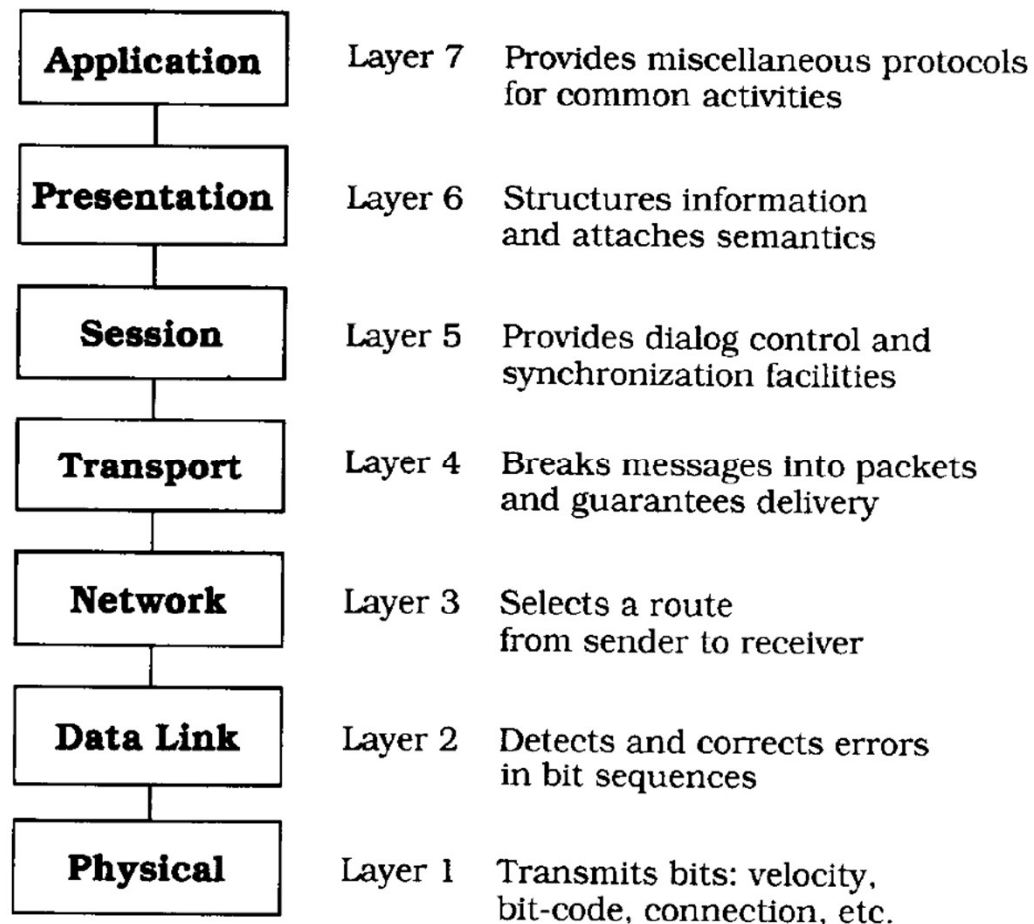
Persistence



Layer

Layers are used to structure an application that can be decomposed into groups of subtasks in which each group of subtasks is at a particular level of abstraction.

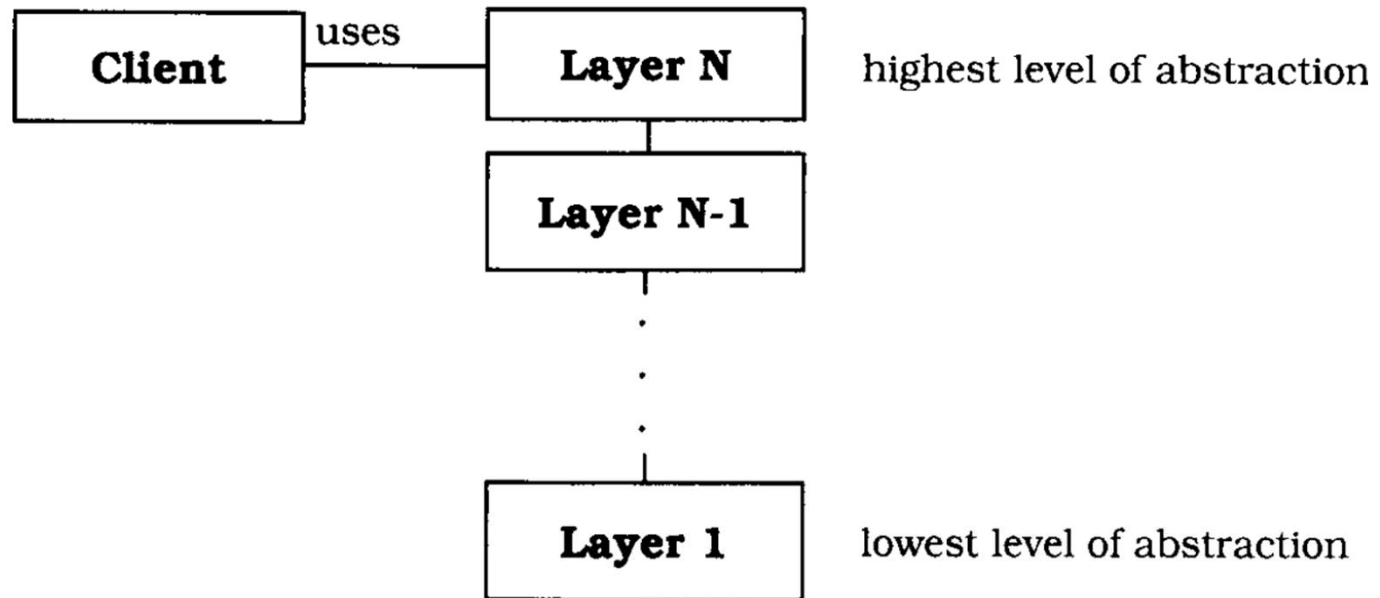
Layer examples



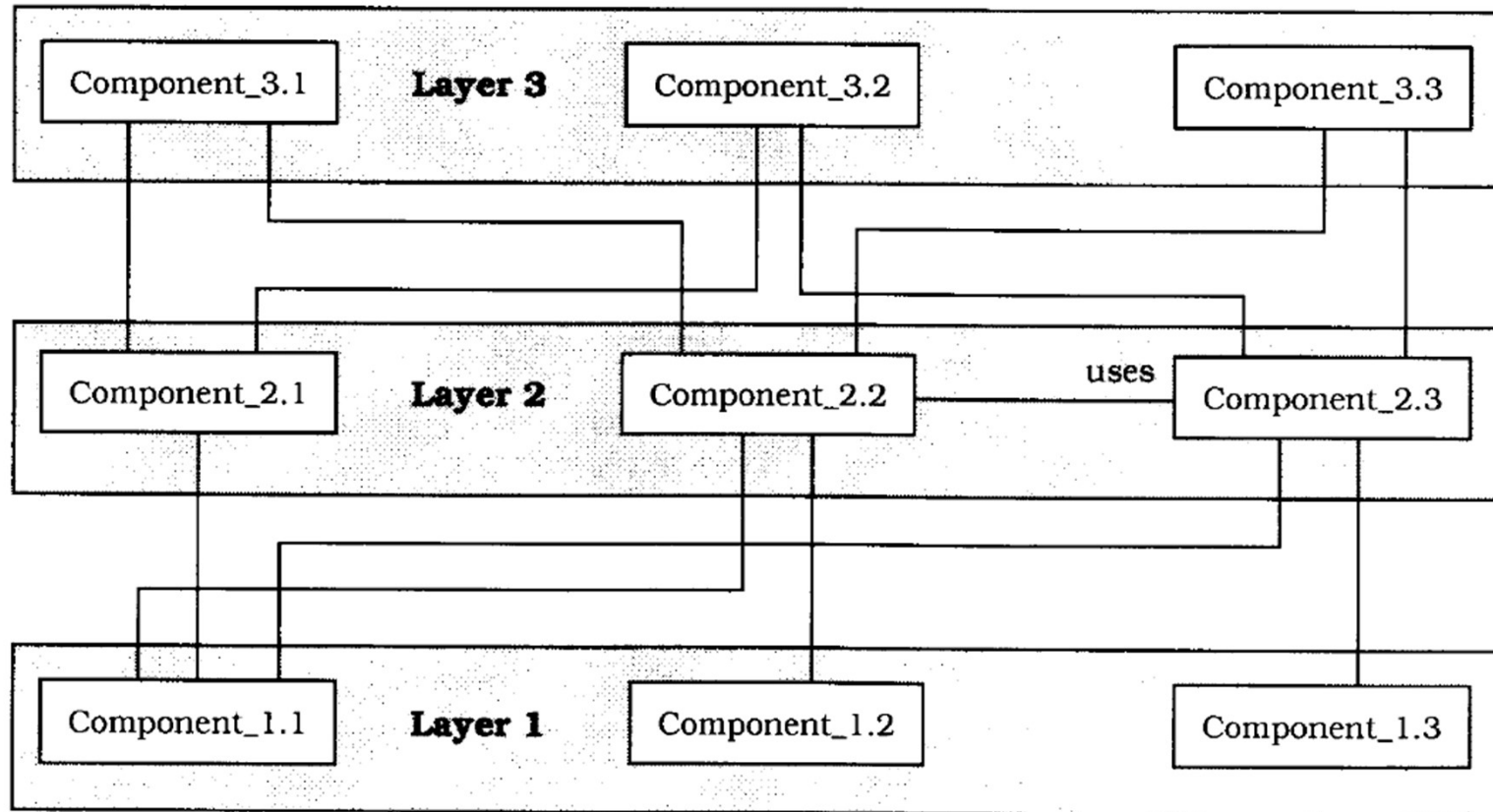
Layer: Structure

Class Layer J	Collaborator • Layer J-1
Responsibility • Provides services used by Layer J+1. • Delegates subtasks to Layer J-1.	

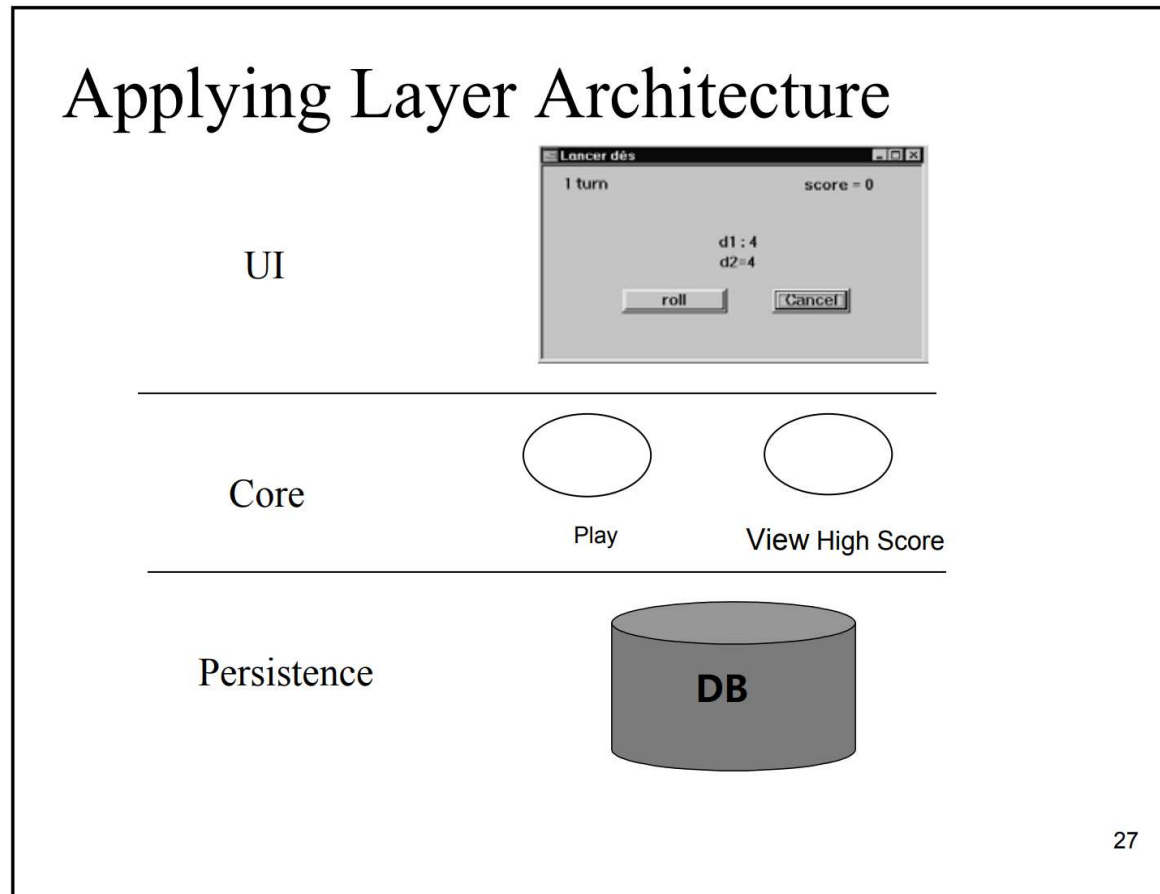
Layer: Structure



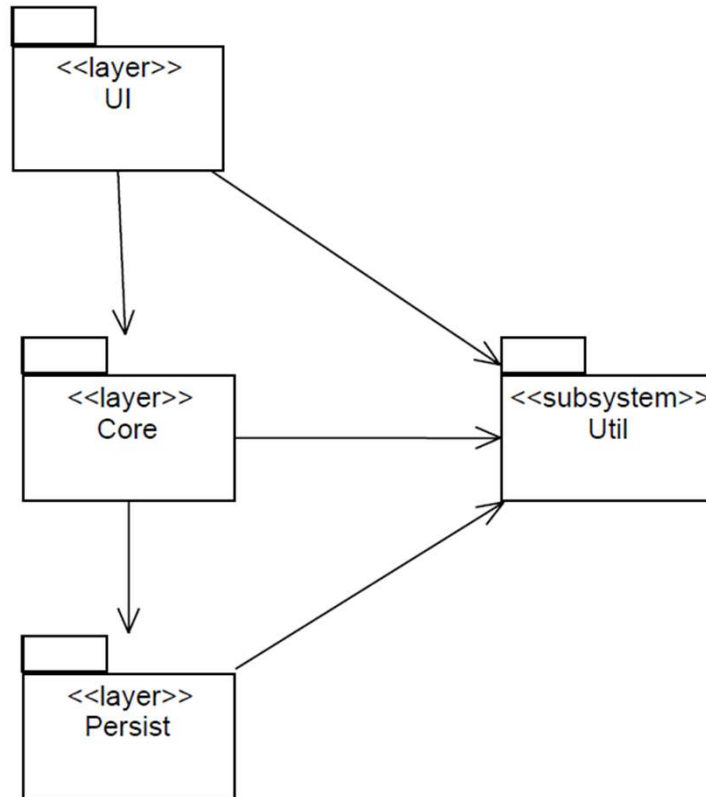
Layer and components....



Applying Layer Architecture



Package decomposition

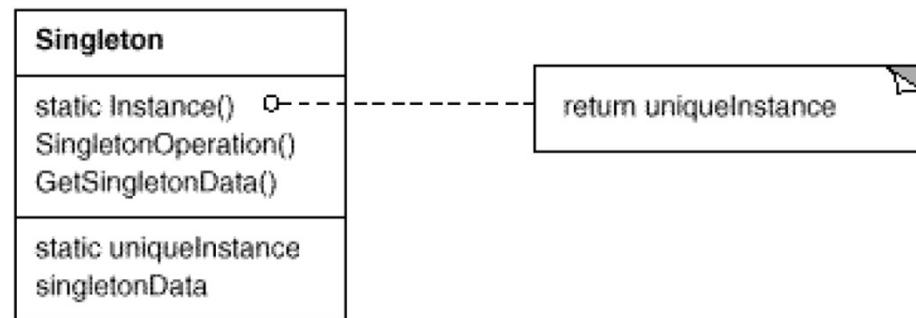


Layer <<core>>

- Contain business logic classes...
- Use of singleton design pattern...

Singleton design pattern

- Ensure a class only has one instance, and provide a global point of access to it.



Singleton structure